

MESB24

Exercise day 2

Have you first tried to answer the questions for yourself?

Stage 0: Preparation stage for ecModel reconstruction

Q1: What would you do if your organism is not at all on UniProt, or only a few of its proteins?

A1: You can manually prepare a file that contains the same type of information as would have been downloaded from UniProt. An example of this can be found in the `tutorials/full_ecModel/data/uniprot.tsv`. The required information you need to gather from other (species-specific?) databases.

Stage 1: Expansion from a conventional GEM to an ecModel structure

Q2: Inspect the `noUniprot` output. This is not empty, what does this mean, and is there something that can be done to resolve it? How does the current output affect the model that we are generating, will it have a large or minor impact?

A2: GECKO reports:

WARNING: No kegg.ID is specified, unable to download KEGG DB.

WARNING: The 7 gene(s) reported in `noUniprot` cannot be found in `data/uniprot.tsv`, these will not be enzyme-constrained. If you intend to use different Uniprot data (e.g. from a different proteome, make sure you first delete the existing `data/uniprot.tsv` file.

The model has 7 genes for which the `uniprot.tsv` file does not contain data (we do not know to how many reactions these are connected: they could all be associated to the same reaction, or maybe each of them is associated to 10 different reactions, although a situation in between is more likely). You could check in the `uniprot.tsv` file, to see if there is no obvious mistake in the file that would prevent GECKO from finding these gene names. Also see if the genes in `noUniprot` have a style that is different from the other genes in the model: maybe a mistake is in the model? Or maybe these refer to heterologous genes? It is also not unlikely that the UniProt parameters in the model adapter are not sufficient to gather the relevant proteins. Instead of 'proteomes', it is sometimes more suitable to refer to a set of UniProt proteins by how they are combined under one 'taxonomy'. If it is just a few proteins that are not matched, then it might be most convenient to just manually add the relevant information to `uniprot.tsv`. With just 7 missing genes, there will not be a very large impact to the model, especially as we can add standard k_{cat} values instead (see later on in the exercise).

Q3: Notice that `ecModel.ec.kcat` has the same length as `ecModel.ec.rxns` and not `ecModel.ec.enzymes`. But k_{cat} values are properties of enzymes, so should they not be defined for each enzyme, instead of each reaction? Clearly not, but why are k_{cat} values here defined per reaction? And can you identify which field in `ecModel.ec` is still connecting enzymes to reactions?

A3: The k_{cat} value is indeed a property of an enzyme, but it is of course also substrate specific. While hexokinase can catalyze the phosphorylation of both glucose and fructose, it will catalyze these reactions at different rates. An example from [BRENDA](#) (compare the wild-type enzymes), with k_{cat} values of hexokinase in *Eimeria tenella*:

TURNOVER NUMBER [1/s] ▲▼	SUBSTRATE ▲▼	ORGANISM ▲▼	UNIPROT ▲▼	COMMENTARY ▲▼	×	LITERATURE ▲▼	IMAGE ▲▼
33.7	2-deoxy-D-glucose	<i>Eimeria tenella</i>	A0A172WCE4	wild-type, pH 9.0, 36°C		739239	
1.5	D-fructose	<i>Eimeria tenella</i>	A0A172WCE4	wild-type, pH 9.0, 36°C		739239	
10.8	D-glucose ☐	<i>Eimeria tenella</i>	A0A172WCE4	mutant N216K, pH 9.0, 36°C		739239	
39.7			A0A172WCE4	mutant D217H, pH 9.0, 36°C		739239	
219			A0A172WCE4	wild-type, pH 9.0, 36°C		739239	
0.025	D-mannose	<i>Eimeria tenella</i>	A0A172WCE4	wild-type, pH 9.0, 36°C		739239	

So while k_{cat} values are reaction-specific, alternative enzymes (or isoenzymes) can have different k_{cat} values for the same reaction. In the table, the mutant enzymes can be seen as “alternative enzymes”, and they have different k_{cat} . This is considered in GECKO by splitting up reactions (done by `makeEcModel`), so that each reaction is only catalyzed by one enzyme (or one enzyme complex).

The enzyme-specific data and reaction-specific data are then connected in the `ecModel` through the `ecModel.ec.rxnEnzMat` matrix. Here a 1 indicate that an enzyme is associated to a particular reaction (and a 0 means no association). If information about the number of subunits in a complex is known, then this matrix could also contain other integers.

Q4: Find the first k_{cat} value from `kcatList_fuzzy` on BRENDA, and describe why origin for this match was 4.

A4: The EC number and `kcat` value to search for are 1.1.2.4 and 1500, as you can find out with `kcatList_fuzzy.eccodes(1)` and `kcatList_fuzzy.kcats(1)`. You end up on [this page](#), and see that 1500 is a k_{cat} value that is measured in *S. cerevisiae* with D-lactate as substrate. Origin code 4 means “any organism, any substrate, `kcat`”, which indeed seems to be the case here. The k_{cat} value is from a different organism, and also the substrate does not match. If you run `kcatList_fuzzy.substrates{1}`, you’ll get that the reaction in the model has (R)-lactate and ferricytochrome c as substrates. The matching happens purely by comparing characters in the metabolite names, so what is actually not considered is that (R)-lactate and D-lactate are the same metabolite! See for instance [here](#). So the true origin number should be 2: “any organism, correct substrate, `kcat`”. It was deemed not worth the effort, trying to account for all such cases.

Q5: What would be the consequence of a missing k_{cat} value for a reaction?

A5: Similar as with missing UniProt data (see Q2), enzyme-constraints cannot be applied to this reaction, as it would require the calculation of MW/k_{cat} . Similar as in Q2, missing a few k_{cat} values will not have a major impact, especially when adding the standard k_{cat} values instead.

Q6: How has the new reaction equation changed after applying k_{cat} constraints?

A6: GECKO reports:

```
(R)-lactate[c] + 2 ferricytochrome c[m] + 0.0021854 prot_M7WMJ7[c] +  
0.01188 prot_M7XVT2[c] => 2 ferrocytochrome c[m] + pyruvate[c]
```

Two additional substrates have appeared: there are two proteins that are both required. Without one of the two proteins, the reaction would not occur, so both proteins are essential subunits of the enzyme complex. The stoichiometry by which they are required is defined by MW/k_{cat} , as is detailed in the GECKO 3 protocol. Note that MW is enzyme-specific, while k_{cat} is reaction specific. Because both proteins are required, there is only one k_{cat} value that is valid for both subunits. DLKcat would be able to predict separate k_{cat} values for each of the two subunits, but `selectKcatValue` will then choose the highest k_{cat} value (by default, you can also select other options) as the one representing the whole reaction.

Q7: What is the consequence of the different f -factor for later model simulations?

A7: The f -factor was reduced from 0.50 to 0.37. The lower f -factor will reduce the size of the protein pool. A smaller fraction of the total cellular protein content are enzymes, and it is only proteins from that fraction that would be able to catalyze the metabolic reactions in the model. Higher f -factor would give the opposite effect.

Q8: What is the correct meaning of the protein pool constraint, and do you know the unit of this constraint (is it mmol/gDCW/h like other reactions, or something else)?

A8: Somewhat confusingly, the protein pool constraint (but also the constraints on individual proteins once proteomics is integrated at a later stage) is not mmol/gDCW/h, as would be the case for pretty much all other reactions in the model. Instead, the unit is mg/gDCW. For a thorough justification for this, you can look at the [Appendix](#) of the original GECKO publication (although pay attention that GECKO 1 and 2 used 1/kcat as stoichiometric coefficients for the protein usage, while GECKO 3 uses MW/kcat. Another demonstration of the unit of the protein pool constraint is how it is calculated: $P_{tot} * f * \sigma$. As f and σ are unitless, it follows the same unit as P_{tot} : mg/gDCW.

Q9: Compare the gene associations in the conventional GEM with the ecModel for reaction r_0001, and describe what has happened.

A9: In the ecModel it is:

'RHTO_06352 and RHTO_05208 '

'RHTO_05208 and RHTO_02645 '

In the conventional GEM it is:

' (RHTO_06352 and RHTO_05208) or (RHTO_05208 and RHTO_02645) '

What we see is what is also mentioned in the answer to Q3: isoenzymes have been split into separate reactions. Here, each isoenzyme consists of two essential subunits, which are still kept together. The effect of this is visible in the answer to Q6: both protein subunits need to be present before the reaction can occur.

Q10: Imagine that `r_0001_EXP_1` carries a flux of 2 mmol/gDCW/h. Describe the flux going through the other reactions (think about the direction [positive, negative] of the flux!).

A10: The output that is shown in GECKO:

Reaction ID	Reaction equation	Lower bound	Upper bound
<code>r_0001_EXP_1</code>	<code>(R)-lactate[c] + 2 ferricytochrome c[m] + 0.0021854 prot_M7WMJ7[c] + 0.01188 prot_M7XVT2[c] => 2 ferrocyanochrome c[m] + pyruvate[c]</code>	0	1000
<code>usage_prot_M7WMJ7</code>	<code>prot_M7WMJ7[c] => prot_pool[c]</code>	-1000	0
<code>usage_prot_M7XVT2</code>	<code>prot_M7XVT2[c] => prot_pool[c]</code>	-1000	0
<code>prot_pool_exchange</code>	<code>prot_pool[c] =></code>	-80.4	0

The flux through `r_0001_EXP_1` will require usage of the two proteins `M7WMJ7` and `M7XVT2`. The first enzyme is required less than the second (0.0021854 vs. 0.01188). As both enzymes have the same k_{cat} in one reaction, this can only be explained by `M7WMJ7` having a lower molecular weight than `M7XVT2`. Then, the two `usage_prot` reactions will carry a flux of -0.0043708 and -0.02376 respectively. The `protein_pool_exchange` reaction will have a flux that is at least -0.0281308 (the sum of the two `usage_prot` fluxes), but because it is highly likely that there are more proteins being used in this simulation, this flux will likely be much more negative. As extreme value, it will not go beyond -80.4.

Q11: Is the protein pool realistic? Consider that the f - and σ -factors have not yet been considered here, if you want to determine the total cellular protein content.

A11: GECKO reports:

At maximum growth rate, a protein pool of 1047 mg/gDCW is required. By itself, this is already not realistic: > 1 g/gDCW is physically impossible. Consider that the f - and σ -factors have not even been considered. In that case, this would imply: $1047 * (1/0.3667) * (1/0.5) = 5710$ mg/gDCW. One gram of dry cell weight contains 5 grams of protein. A miracle.

Q12: Comment on the enzyme usage of M7WT33: is the percUsage realistic? What can be the reason for this?

A12: Only showing output from the first entry (and swapped orientation of the table):

protID	M7WT33
geneID	RHT0_02306
absUsage	67.0804
percUsage	83.4378
kcat	0.0053667
source	brenda
rxnID	r_0362_EXP_2
rxnNames	dolichyl-phosphate-mannose--protein mannosyltransferase
grRules	RHT0_02306

It is not realistic to suggest that one enzyme takes up 83% of the total enzyme allocation. To convert this to % of total cellular protein content, you only need to consider the f-factor, (not the sigma-factor, as all enzymes will scale with the same amount), which results in: $83.4378 * 0.3667 = 30.5966$, so more than 30% of all protein in the cell. This is not realistic.

Q13: The model can reach the growth rate, but how can you know that the percentage usage is now realistic?

A13: GECKO reports a table with top percentage usages ranging from 19.6-2.99% (will not show the whole table here due to space constraint).

From the perspective of being able to replicate the maximum growth rate, there is no argument why the percentage usages would not be realistic. The best way to find out if it is realistic, would be to compare with proteomics data. Does 5'-phosphoribosylformyl glycinamide synthetase take up almost 20% of all enzymes? Theoretically there is no argument why this could not be the case, but once you have looked at some proteomics data from your organism, you might find out that this is not true. Even looking at proteomics data of related organisms will likely give you a somewhat reasonable estimate whether certain enzymes would take up such a large fraction of the total protein content.

From such a table of top used enzymes, it can be useful to systematically check the underlying support for each of the kcat values. Are they taken from the same organism with the same substrate? Or are there obvious doubts, because the value appears particularly low?

Q14: Inspect what happened with the k_{cat} value and how this will affect the protein usage when the reaction is active.

A14: The k_{cat} value has increased, as was the intention. Also, the `ecModel.ec.source` field now clarifies that we have manually set this k_{cat} value. If you want, you can look at `ecModel.ec.notes(enxIdx)` to see that the original k_{cat} value and source are kept.

When inspecting the reaction equation, the stoichiometric coefficient of the protein usage has decreased from 818.4056 to 8.075. If the reaction would have the same flux, both before and after we changed the k_{cat} value, it would require >100-fold less protein usage with the reduced k_{cat} value. More of the total protein can then be allocated to other proteins, which will allow for a high growth rate overall.

Q15: Describe how it seems like k_{cat} tuning and sigma-factor fitting are trying to achieve opposite goals.

A15: The k_{cat} tuning had as objective to reach a higher maximum growth rate, while the sigma-factor fitting here has the consequence that the maximum growth rate decreases. The k_{cat} tuning did overshoot the objective growth rate, as in each iteration one k_{cat} value was increased by a 10-fold. It is not unlikely (or even: quite likely!) that the 10-fold increase was higher than was required. In particular considering the last enzyme that had its k_{cat} increased: a 1.1-fold increase would perhaps have been sufficient for the model to reach the target growth rate. By instead increasing the k_{cat} value by 10-fold by default, the model was now able to reach an even higher growth rate.

This can be somewhat counteracted by setting an alternative `foldChange` when running `sensitivityTuning`. This will result in more iterations, but smaller steps in k_{cat} tuning.

Sigma-factor fitting tries out all 100 equidistant values between 0.01 and 1, which is a much more fine-grained approach. The resulting model is therefore more closely limited in its growth to the provided experimentally measured maximum growth rate.

Q16: Compare with the table that you gathered around Q10. What changes have occurred? What will this have for consequence? Look again at your answer of Q10 and see if this would be affected.

A16: The output that is shown in GECKO:

Reaction ID	Reaction equation	Lower bound	Upper bound
r_0001_EXP_1	(R)-lactate[c] + 2 ferricytochrome c[m] + 0.0021854 prot_M7WMJ7[c] + 0.01188 prot_M7XVT2[c] => 2 ferrocytochrome c[m] + pyruvate[c]	0	1000
usage_prot_M7WMJ7	prot_M7WMJ7[c] => prot_pool[c]	-0.23353	0
usage_prot_M7XVT2	prot_M7XVT2[c] => prot_pool[c]	-0.01236	0
prot_pool_exchange	prot_pool[c] =>	-41.806	0

The lower bounds of the usage_prot reactions are now set to proteomics data, they are not longer unconstrained (at -1000). The change in the prot_pool_exchange reaction lower bound is **not** a results of constrainEnzConcs. If you look back through the script, then at Step 51A, Q15, sigmaFitter was run. By changing the sigma-factor, this affected the size of the protein pool.

Q17: As you did for Q10, again imagine that r_0001_EXP_1 would carry a flux of 2 mmol/gDCW/h. Describe the flux going through the other reactions (think about the direction [positive, negative] of the flux!) and compare with your answer in Q10.

A17: In this case, r_0001_EXP_1 cannot carry a flux of 20 mmol/gDCW/h! If we look at the answer to Q10, there we found out that the usage_prot reactions would carry fluxes of -0.0043708 and -0.02376, for M7WMJ7 and M7XVT2 respectively. But after constraining with enzyme concentrations, the new lower bounds will not allow such fluxes through all of the required usage_prot reactions. In particular, the concentration of M7XVT2 is much lower: 0.01236 g/gDCW vs. the required 0.02376 g/gDCW. This will therefore be the protein that is constraining the reaction. The maximum flux that can go through r_0001_EXP_1, will be: $0.01236 / 0.01188 = 1.04$ mmol/gDCW/h, not the 2 mmol/gDCW/h that is mentioned in the question. The flux through the usage_prot reactions will then be 0.002274 and 0.01236 respectively. This implies that all of M7XVT2 is used (its usage_prot reaches the lower bound), while for M7WMJ7 there is still spare capacity. Of the available 0.23253 g/gDCW of M7WMJ7, 0.002274 g/gDCW will be used by r_0001_EXP_1. $(0.002274 / 0.23253) \times 100\% = 0.98\%$ of available M7WMJ7 is used.

Q18: Find the enzyme that was reported as potentially problematic. Why do you think this was one singled out? Do you think this is really the most problematic enzyme, or could you also select a different enzyme as more problematic?

A18: Partial output from GECKO:

uniprotIDs	oldConcs	flexConcs	frequence	ratioIncr
M7WJQ7	0.0011658	0.62001	1	531.84
M7WGT6	0.0034593	0.67048	1	193.82
M7WFB4	0.011036	1.323	6	119.88
M7WL39	0.010504	1.2227	1	116.4

M7WFB4 is reported as potentially problematic. This is the enzyme that has been flexibilized in the most iterations (6 times). At the same time, its concentration was “only” increased 66-fold. The enzyme M7WJQ7 on the other hand was only flexibilized once, but at that point its concentration was increased 531-fold. It seems a bit arbitrary to suggest that M7WFB4 is the problematic protein concentration here.

Q19: Are the top-flexibilized enzymes and the ratios by which they increase the same in the two tables? Can you come up with some criticism of how sensitivityTuning works?

A19: Partial output from GECKO:

uniprotIDs	oldConcs	flexConcs	frequence	ratioIncr
M7WJQ7	0.0011658	3.2211	3	2763
M7X6W8	0.013761	1.9811	1	143.96
M7WGT6	0.0034593	0.38355	1	110.87
M7XNE0	0.0069309	0.42624	1	61.499

The list can be quite different. M7WJQ7 is still on top, but now increased 3 times by 2763-fold. Keep in mind that now the model can reach the growth rate of 0.18, while in the previous table it only reached 0.067 (according to `flexibilizeEnzConcs`). At the same time, not all enzymes require higher increases. M7WFB4 that increased almost 120-fold previously, now only requires almost 30-fold increase, even though a higher growth rate was reached. Is there some randomness in the flexibilization approach? No, but these results show that it might not be a good idea to only look at the top controlling enzyme and flexibilize its concentration, before another round of flexibilization. But we’re working on an improved function for this, that’s considers flexibilization of all enzyme concentrations at the same time, stay tuned!

Q20: Why would the minimum protein usage be zero if you do not set a minimum growth rate?

A20: If the model does not have to produce any biomass (or have a minimum flux through another flux, maybe biosynthesis of your product of interest?), then the most optimal solution is the situation where all fluxes are zero. If no reactions have any flux, there is also no need for any enzyme.

Q21: Here, the '1' in the `setParam` command means minimization of protein pool usage, why?

A21: As we've seen in previous questions, the direction of the `prot_pool_exchange` reaction is defined so that it will always have a negative (or zero) flux. This negative flux means that `prot_pool` is used by the model. When we talk about "minimizing protein pool usage", we basically mean that we want a protein pool usage that is as close as possible to zero. With the reaction defined in the opposite direction, this practically means that the FBA should try to maximize the flux through the `prot_pool_exchange` reaction instead. A maximized negative flux is zero.

Q22: Why is it not possible to directly compare the two FBA outputs, which were stored at `sol.x` (from the `ecModel`) and `solGEM.x` (from the conventional GEM)?

A22: The `ecModel` contains many more reactions, because of splitting reversible reactions and keeping isoenzymes separate, as well as including `usage_prot` reactions. The FBA outputs from the two models are therefore also different: both are vectors of reaction fluxes, but of different length (because more reactions), and the position of reactions on these vectors might not match.

Q23: There is a problem with these fluxes. The models were supposedly fitted to experimental flux data, but does the FBA result from the ecModel really show this? The problem does not seem to exist in the conventional GEM.

A23: Output from GECKO:

Reaction ID	Reaction name	GEM	ecModel
r_1672	(carbon dioxide exchange):	6.6662	6.7835
r_1714	(D-glucose exchange):	-2.3098	-2.489
r_1761	(ethanol exchange):	0	0.38971
r_1808	(glycerol exchange):	0.0441	0.0441
r_1832	(H ⁺ exchange):	-0.41748	-0.27909
r_1861	(iron(2+) exchange):	-1.8e-07	-1.7775e-07
r_1992	(oxygen exchange):	-5.6857	-5.4585
r_2005	(phosphate exchange):	-0.039503	-0.038992
r_2056	(succinate exchange):	0	0.066561
r_2060	(sulphate exchange):	-0.015302	-0.015111
r_2091	(urea exchange):	-0.36983	-0.36521
r_2100	(water exchange):	8.206	7.9803
r_2111	(growth):	0.18	0.17775
r_1672	(carbon dioxide exchange):	6.6662	6.7835

In the ecModel, there is ethanol and succinate secretion, while this is absent in the GEM. Also in the experimental data (in `fluxData.tsv`), there is no ethanol nor succinate secretion reported. It is possible that this is a problem with the experimental data: ethanol and succinate are excreted but just have not been measured. If ethanol and succinate are truly not secreted, then the problem should be found in the model. A potential explanation is that one of the integrated enzyme concentrations are preventing a more efficient metabolism. A good strategy would be to look at top used enzymes (high capacity), and see if any of those are related to ethanol and/or succinate production. Another explanation is that one/multiple k_{cat} values are incorrect. For the sake of this exercise, we will not go into further detail on how to resolve this issue.

Q24: Look at the dimensions of `minFlux` and `maxFlux`: they are of width 3, but what is their length? How is it possible that this contain the fluxes from all three models, including the ecModels?

A24: The `minFlux` and `maxFlux` have the same length as the number of reactions in the conventional GEM. For the ecModels this means that the reaction fluxes have already been mapped back to the conventional GEM (so `mapRxnsToConv` runs as part of `ecFVA`).

Q25: Describe what you see in the ecFVA plot.

A25: The flux variability of reactions in three different models, plotted as cumulative distribution. From the legend it is already clear that the median flux variability is significantly lower in model #3 -> the proteomics integrated ecModel. Looking at the lines, models #2 and #3 (the two ecModels) have virtually no reactions with a variability higher than 3 (i.e. $3 \cdot 10^0$ on the x-axis), as these lines reach 1 on the y-axis. In contrast, the conventional GEM has 5% of reactions that have a variability higher than 1000. Note that these models are all constrained with experimentally measured reaction rates from bioreactor cultivations. Based on these measured rates, we can get much more reliable estimation of all other metabolic rates by using an ecModel.